

```
"""
```

```
GDM Break Linkage Diagram Generator + Journal Posting Analysis
```

```
=====
```

```
Two outputs:
```

1. Interactive HTML network diagram (mirrors your Bonds linkage image)
for ALL GDM asset classes – colour-coded exactly as your diagram:
 GREEN = Reconciliation (rec node)
 BROWN = Product
 BLUE = Break Type + Account Group
 ORANGE = Team
 WHITE = Asset class / root node
2. Journal posting analysis – for each rec break in the current MI
report, shows how many journals are posted irrespective of root cause,
and estimates how many could be suppressed if root cause is known.

```
Run:
```

```
pip install pandas numpy pyvis networkx plotly openpyxl  
python gdm_linkage_diagram.py
```

```
# or with your actual data:
```

```
# set DF_PATH below and uncomment the pd.read_csv line
```

```
"""
```

```
from __future__ import annotations  
import pandas as pd  
import numpy as np  
import os  
from collections import defaultdict
```

```
# — Try to import visualisation libs
```

```
try:
```

```
    from pyvis.network import Network  
    PYVIS_OK = True
```

```
except ImportError:
```

```
    PYVIS_OK = False
```

```
    print("[WARN] pyvis not installed – HTML diagram will be skipped. pip install
```

```
try:
```

```
    import plotly.graph_objects as go  
    import plotly.express as px  
    PLOTLY_OK = True
```

```
except ImportError:
```

```
    PLOTLY_OK = False
```

```

    print("[WARN] plotly not installed - bar charts will be skipped. pip install

# =====
# SECTION 1 - CONFIGURATION
# =====

DF_PATH = None          # Set to "your_file.csv" or "your_file.parquet"
OUTPUT_DIR = "gdm_output"
os.makedirs(OUTPUT_DIR, exist_ok=True)

# Column name mapping - adjust to match your actual MI report headers
COL = {
    "date":          "Date",
    "entity":        "Entity",
    "asset_class":    "Asset Class",
    "product":        "High Level Product",
    "trade_ref":      "TRADE REF",
    "rec_name":       "Rec Name (as per Rec Cube)",
    "break_amount":   "BREAK AMOUNT GBP",
    "break_type":     "Break Type",          # e.g. Principal/Position, Cash, MTM/Ac
    "account_group":  "Account Group",       # e.g. BS, PnL
    "team":           "Team",               # e.g. Rates, Credits, EMNY
    "ccy":            "CCY",
    "journal_ref":    "Journal Reference",   # optional - set None if absent
}

# =====
# SECTION 2 - GDM REC UNIVERSE
# Full precedence table from your image, enriched with:
#   - asset_classes: which GDM products flow through this rec
#   - break_types:   break types observed in your Bonds diagram pattern
#   - teams:         owning teams (orange boxes in your diagram)
#   - tlm_stage:     from your precedence table column E
# =====

GDM_RECS = [

    # --- PRECEDENCE 1: FO/FO - Capture / population consistency ---
    {
        "rec":          "BBG vs XTP - Futures",
        "precedence":    1,
        "divide":        "FO/FO",
        "frequency":     "Weekly",
        "tlm_stage":     "Capture/population consistency (FO layer)",
        "asset_classes": ["Futures", "ETD"],
        "break_types":   ["Principal/Position"],
        "account_groups": ["BS"],
    }

```

```

    "teams":      ["Rates", "Credits"],
    "rank":       10,
},
{
    "rec":        "GC vs Bloomberg - Credit Linked Notes",
    "precedence": 1,
    "divide":     "FO/FO",
    "frequency":  "Weekly",
    "tlm_stage":  "Capture/population consistency (FO layer)",
    "asset_classes":["Credit Linked Notes", "Credits"],
    "break_types": ["Principal/Position"],
    "account_groups":["BS"],
    "teams":      ["Credits"],
    "rank":       11,
},
{
    "rec":        "Summit vs XTP - Futures (MTM)",
    "precedence": 1,
    "divide":     "FO/FO",
    "frequency":  "Weekly",
    "tlm_stage":  "Capture/population consistency (FO layer)",
    "asset_classes":["Futures", "MTM"],
    "break_types": ["MTM/Accrual"],
    "account_groups":["PnL"],
    "teams":      ["Rates"],
    "rank":       12,
},

# — PRECEDENCE 2: FO/ARE — Sub-ledger / accounting engine alignment —
{
    "rec":        "RIMS vs SPARC Reconciliation - Sub-ledger",
    "precedence": 2,
    "divide":     "FO/ARE",
    "frequency":  "Daily",
    "tlm_stage":  "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["Bonds", "MTN", "Credits", "Rates"],
    "break_types": ["Principal/Position"],
    "account_groups":["BS"],
    "teams":      ["Rates", "Credits", "Bonds"],
    "rank":       20,
},
{
    "rec":        "BBG vs GCA - EM Bonds",
    "precedence": 2,
    "divide":     "FO/ARE",
    "frequency":  "Weekly",
    "tlm_stage":  "FO ↔ ARE feed alignment (sub-ledger / accounting engine)"

```

```

    "asset_classes":["Bonds", "EM Bonds"],
    "break_types": ["Cash", "MTM/Accrual"],
    "account_groups":["PnL"],
    "teams":        ["EMNY", "Credits"],
    "rank":          21,
},
{
    "rec":           "BBG vs RIMS/SPARC - Bonds (MTM)",
    "precedence":    2,
    "divide":        "FO/ARE",
    "frequency":     "Weekly",
    "tlm_stage":     "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["Bonds"],
    "break_types":  ["Principal/Position", "Cash"],
    "account_groups":["BS"],
    "teams":         ["Rates", "Bonds"],
    "rank":          22,
},
{
    "rec":           "Murex vs Sparc - FXO IRD (Cash, MTM)",
    "precedence":    2,
    "divide":        "FO/ARE",
    "frequency":     "Weekly",
    "tlm_stage":     "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["FX Options", "IRD"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":         ["Rates"],
    "rank":          23,
},
{
    "rec":           "NCOMs vs Treats - Collateral (Position, Interest Accrual",
    "precedence":    2,
    "divide":        "FO/ARE",
    "frequency":     "Weekly",
    "tlm_stage":     "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["Collateral", "Repo"],
    "break_types":  ["Principal/Position", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":         ["Rates"],
    "rank":          24,
},
{
    "rec":           "Sentry vs LoanIQ - Secondary Loans",
    "precedence":    2,
    "divide":        "FO/ARE",
    "frequency":     "Weekly",

```

```

    "tlm_stage":      "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes": ["Loans", "Credits"],
    "break_types":   ["Principal/Position", "Cash"],
    "account_groups": ["BS"],
    "teams":         ["Credits"],
    "rank":          25,
},
{
    "rec":            "Summit vs Sparc - IRD MTM",
    "precedence":     2,
    "divide":         "FO/ARE",
    "frequency":      "Weekly",
    "tlm_stage":      "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes": ["IRD"],
    "break_types":   ["MTM/Accrual"],
    "account_groups": ["PnL"],
    "teams":         ["Rates"],
    "rank":          26,
},
{
    "rec":            "Summit/XFOS vs SPARC - MTNs MTM rec (Rates/Credits)",
    "precedence":     2,
    "divide":         "FO/ARE",
    "frequency":      "Weekly",
    "tlm_stage":      "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes": ["MTN", "Rates", "Credits"],
    "break_types":   ["Principal/Position", "MTM/Accrual"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Rates", "Credits"],
    "rank":          27,
},
{
    "rec":            "GC vs DTP - CDS - Credit Centralised",
    "precedence":     2,
    "divide":         "FO/ARE",
    "frequency":      "Bi-Weekly",
    "tlm_stage":      "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes": ["CDS", "Credits"],
    "break_types":   ["Principal/Position", "MTM/Accrual"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Credits"],
    "rank":          28,
},
{
    "rec":            "Martini vs SPARC - Repo (MTM, Principal, BvB Fees)",
    "precedence":     2,
    "divide":         "FO/ARE",

```

```

    "frequency":    "Bi-Weekly",
    "tlm_stage":    "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes":["Repo"],
    "break_types":  ["Principal/Position", "MTM/Accrual", "Cash"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Rates"],
    "rank":         29,
},
{
    "rec":          "Summit vs Treats - FX (MTM, Cash)",
    "precedence":   2,
    "divide":       "FO/ARE",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes":["FX"],
    "break_types":  ["MTM/Accrual", "Cash"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Rates"],
    "rank":         30,
},
{
    "rec":          "Summit vs Treats - MM (MTM, Notional)",
    "precedence":   2,
    "divide":       "FO/ARE",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes":["Money Market", "MM"],
    "break_types":  ["Principal/Position", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Rates"],
    "rank":         31,
},
{
    "rec":          "Summit/XFOS vs SPARC - MTNs Cash rec (Rates/Credits)",
    "precedence":   2,
    "divide":       "FO/ARE",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "FO ↔ ARE feed alignment (sub-ledger / accounting engine)
    "asset_classes":["MTN", "Rates", "Credits"],
    "break_types":  ["Cash"],
    "account_groups":["BS"],
    "teams":        ["Rates", "Credits"],
    "rank":         32,
},
{
    "rec":          "XFOS vs DTP - TRS (MTM, Cash)",
    "precedence":   2,

```

```

    "divide":      "FO/ARE",
    "frequency":   "Bi-Weekly",
    "tlm_stage":   "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["TRS", "Credits"],
    "break_types": ["MTM/Accrual", "Cash"],
    "account_groups":["BS", "PnL"],
    "teams":       ["Credits"],
    "rank":        33,
},
{
    "rec":         "XFOS vs Treats - EM SWAPS (MTM, Cash Rec)",
    "precedence":  2,
    "divide":      "FO/ARE",
    "frequency":   "Bi-Weekly",
    "tlm_stage":   "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["EM Swaps", "Rates"],
    "break_types": ["MTM/Accrual", "Cash"],
    "account_groups":["BS", "PnL"],
    "teams":       ["Rates", "EMNY"],
    "rank":        34,
},
{
    "rec":         "BvB Fees Rec",
    "precedence":  2,
    "divide":      "FO/ARE",
    "frequency":   "Monthly",
    "tlm_stage":   "FO ↔ ARE feed alignment (sub-ledger / accounting engine)",
    "asset_classes":["Bonds", "Credits", "Rates"],
    "break_types": ["Cash"],
    "account_groups":["PnL"],
    "teams":       ["Credits", "Rates"],
    "rank":        35,
},

# — PRECEDENCE 3: ARE/BO - Settlement / control / contra —————
{
    "rec":         "BBG/Sophis/GC Sparc vs Rims/Sparc - Bond Control",
    "precedence":  3,
    "divide":      "ARE/BO",
    "frequency":   "Daily",
    "tlm_stage":   "Settlement / control / contra (ARE/BO)",
    "asset_classes":["Bonds"],
    "break_types": ["Principal/Position", "Cash"],
    "account_groups":["BS"],
    "teams":       ["Bonds", "Rates"],
    "rank":        40,
},

```

```

{
  "rec":          "DTP vs HUB - CRD Contra",
  "precedence":   3,
  "divide":       "ARE/BO",
  "frequency":    "Daily",
  "tlm_stage":    "Settlement / control / contra (ARE/BO)",
  "asset_classes":["CDS", "Credits"],
  "break_types":  ["Cash"],
  "account_groups":["BS"],
  "teams":        ["Credits"],
  "rank":         41,
},
{
  "rec":          "GCA vs DTP - CRD Control",
  "precedence":   3,
  "divide":       "ARE/BO",
  "frequency":    "Daily",
  "tlm_stage":    "Settlement / control / contra (ARE/BO)",
  "asset_classes":["CDS", "Credits"],
  "break_types":  ["Cash", "Principal/Position"],
  "account_groups":["BS"],
  "teams":        ["Credits"],
  "rank":         42,
},
{
  "rec":          "HUB vs Summit/Sparc - IRD Contra",
  "precedence":   3,
  "divide":       "ARE/BO",
  "frequency":    "Daily",
  "tlm_stage":    "Settlement / control / contra (ARE/BO)",
  "asset_classes":["IRD"],
  "break_types":  ["Cash"],
  "account_groups":["BS"],
  "teams":        ["Rates"],
  "rank":         43,
},
{
  "rec":          "Martini/Sparc vs Rims/Sparc - Repo Control",
  "precedence":   3,
  "divide":       "ARE/BO",
  "frequency":    "Daily",
  "tlm_stage":    "Settlement / control / contra (ARE/BO)",
  "asset_classes":["Repo"],
  "break_types":  ["Cash", "Principal/Position"],
  "account_groups":["BS"],
  "teams":        ["Rates"],
  "rank":         44,
}

```



```

},
{
    "rec":          "RIMS/Sparc vs HUB Cash Contra Rec",
    "precedence":   3,
    "divide":       "ARE/BO",
    "frequency":    "Daily",
    "tlm_stage":    "Settlement / control / contra (ARE/BO)",
    "asset_classes":["Bonds", "MTN", "Credits"],
    "break_types":  ["Cash"],
    "account_groups":["BS"],
    "teams":        ["Rates", "Credits"],
    "rank":         45,
},
{
    "rec":          "SPARC Netted reconciliation",
    "precedence":   3,
    "divide":       "ARE/BO",
    "frequency":    "Daily",
    "tlm_stage":    "Settlement / control / contra (ARE/BO)",
    "asset_classes":["Bonds", "MTN", "Credits", "Rates", "Repo"],
    "break_types":  ["Cash", "Principal/Position"],
    "account_groups":["BS"],
    "teams":        ["Rates", "Credits"],
    "rank":         46,
},

# — PRECEDENCE 4: BO/ARE — Fees / interest posting control —————
{
    "rec":          "HUB vs Mainframe - Fees & Interest",
    "precedence":   4,
    "divide":       "BO/ARE",
    "frequency":    "Daily",
    "tlm_stage":    "Fees / interest posting control (BO↔ARE)",
    "asset_classes":["Bonds", "MTN", "Repo", "IRD", "Credits"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["PnL", "BS"],
    "teams":        ["Rates", "Credits"],
    "rank":         50,
},

# — PRECEDENCE 5: FO/GL — P&L / FTP / GL downstream alignment —————
{
    "rec":          "GC to GCA to BBG - POS + MtM",
    "precedence":   5,
    "divide":       "FO/GL",
    "frequency":    "Weekly",
    "tlm_stage":    "P&L / FTP / GL alignment (downstream)",

```

```

    "asset_classes":["Bonds", "EM Bonds", "Credits"],
    "break_types":  ["MTM/Accrual", "Cash"],
    "account_groups":["PnL"],
    "teams":        ["Credits", "Rates EM LATAM"],
    "rank":          60,
},
{
    "rec":           "HLR - Credit Flow",
    "precedence":    5,
    "divide":        "FO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "P&L / FTP / GL alignment (downstream)",
    "asset_classes":["Bonds", "Credits", "MTN"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["PnL", "BS"],
    "teams":         ["Credits"],
    "rank":          61,
},
{
    "rec":           "HLR - SIP",
    "precedence":    5,
    "divide":        "FO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "P&L / FTP / GL alignment (downstream)",
    "asset_classes":["Bonds", "Credits", "IRD"],
    "break_types":  ["Cash"],
    "account_groups":["PnL"],
    "teams":         ["Credits", "Rates"],
    "rank":          62,
},
{
    "rec":           "HLR Rates",
    "precedence":    5,
    "divide":        "FO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "P&L / FTP / GL alignment (downstream)",
    "asset_classes":["Rates", "IRD", "Repo"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["PnL"],
    "teams":         ["Rates"],
    "rank":          63,
},
{
    "rec":           "Murex vs FTP (Balance Sheet, PL) - FX Options",
    "precedence":    5,
    "divide":        "FO/GL",
    "frequency":     "Bi-Weekly",

```

```

    "tlm_stage":      "P&L / FTP / GL alignment (downstream)",
    "asset_classes": ["FX Options"],
    "break_types":   ["MTM/Accrual", "Cash"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Rates"],
    "rank":           64,
},
{
    "rec":            "SOPHIS vs FTP - Convertible Bonds",
    "precedence":     5,
    "divide":         "FO/GL",
    "frequency":      "Bi-Weekly",
    "tlm_stage":      "P&L / FTP / GL alignment (downstream)",
    "asset_classes": ["Convertible Bonds"],
    "break_types":   ["MTM/Accrual", "Cash"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Credits"],
    "rank":           65,
},
{
    "rec":            "P&L Rec - FXO",
    "precedence":     5,
    "divide":         "FO/GL",
    "frequency":      "Monthly",
    "tlm_stage":      "P&L / FTP / GL alignment (downstream)",
    "asset_classes": ["FX Options"],
    "break_types":   ["MTM/Accrual"],
    "account_groups": ["PnL"],
    "teams":         ["Rates"],
    "rank":           66,
},
{
    "rec":            "P&L Rec - REPO",
    "precedence":     5,
    "divide":         "FO/GL",
    "frequency":      "Monthly",
    "tlm_stage":      "P&L / FTP / GL alignment (downstream)",
    "asset_classes": ["Repo"],
    "break_types":   ["MTM/Accrual", "Cash"],
    "account_groups": ["PnL"],
    "teams":         ["Rates"],
    "rank":           67,
},

```

— PRECEDENCE 6: BO/GL — Accounting / GL integrity —————

```

{
    "rec":            "CRI COMPTA Rec - CDS",

```

```

    "precedence":    6,
    "divide":        "BO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "Accounting / GL integrity (BO→GL)",
    "asset_classes": ["CDS", "Credits"],
    "break_types":   ["Cash", "MTM/Accrual"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Credits"],
    "rank":          70,
},
{
    "rec":           "CRI COMPTA Rec - ETD CC",
    "precedence":    6,
    "divide":        "BO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "Accounting / GL integrity (BO→GL)",
    "asset_classes": ["ETD", "Futures"],
    "break_types":   ["Cash", "MTM/Accrual"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Rates", "Credits"],
    "rank":          71,
},
{
    "rec":           "CRI COMPTA Rec - FXO",
    "precedence":    6,
    "divide":        "BO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "Accounting / GL integrity (BO→GL)",
    "asset_classes": ["FX Options"],
    "break_types":   ["Cash", "MTM/Accrual"],
    "account_groups": ["BS", "PnL"],
    "teams":         ["Rates"],
    "rank":          72,
},
{
    "rec":           "CRI COMPTA Rec - MMK",
    "precedence":    6,
    "divide":        "BO/GL",
    "frequency":     "Weekly",
    "tlm_stage":     "Accounting / GL integrity (BO→GL)",
    "asset_classes": ["Money Market", "MM"],
    "break_types":   ["Cash"],
    "account_groups": ["BS"],
    "teams":         ["Rates"],
    "rank":          73,
},
{

```

```

    "rec":          "CRI COMPTA Rec - OTC CC",
    "precedence":   6,
    "divide":       "BO/GL",
    "frequency":    "Weekly",
    "tlm_stage":    "Accounting / GL integrity (BO→GL)",
    "asset_classes":["OTC", "IRD", "Credits"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Rates", "Credits"],
    "rank":         74,
},
{
    "rec":          "CRI COMPTA Rec - Bonds",
    "precedence":   6,
    "divide":       "BO/GL",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "Accounting / GL integrity (BO→GL)",
    "asset_classes":["Bonds", "MTN"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Bonds", "Rates"],
    "rank":         75,
},
{
    "rec":          "CRI COMPTA Rec - Derivatives",
    "precedence":   6,
    "divide":       "BO/GL",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "Accounting / GL integrity (BO→GL)",
    "asset_classes":["IRD", "FX", "OTC"],
    "break_types":  ["Cash", "MTM/Accrual"],
    "account_groups":["BS", "PnL"],
    "teams":        ["Rates", "Credits"],
    "rank":         76,
},
{
    "rec":          "CRI COMPTA Rec - Margin Call",
    "precedence":   6,
    "divide":       "BO/GL",
    "frequency":    "Bi-Weekly",
    "tlm_stage":    "Accounting / GL integrity (BO→GL)",
    "asset_classes":["OTC", "Margin Call", "IRD"],
    "break_types":  ["Cash"],
    "account_groups":["BS"],
    "teams":        ["Rates", "Credits"],
    "rank":         77,
},

```

```

{
  "rec":          "CRI COMPTA Rec - REPO",
  "precedence":   6,
  "divide":       "BO/GL",
  "frequency":    "Bi-Weekly",
  "tlm_stage":    "Accounting / GL integrity (BO→GL)",
  "asset_classes":["Repo"],
  "break_types":  ["Cash", "MTM/Accrual"],
  "account_groups":["BS", "PnL"],
  "teams":        ["Rates"],
  "rank":         78,
},
]

# Build lookup
REC_META = {r["rec"]: r for r in GDM_RECS}
REC_RANK  = {r["rec"]: r["rank"] for r in GDM_RECS}

# =====
# SECTION 3 - PROPAGATION EDGES
# Derived from your Bonds diagram pattern + precedence order
# Every edge: (from_rec, to_rec, product, break_type, account_group)
# =====

PROPAGATION_EDGES = [

  # — BONDS chain (your proven diagram) —————
  ("RIMS vs SPARC Reconciliation - Sub-ledger",
   "BBG vs GCA - EM Bonds",
   "Bonds", "Principal/Position", "BS"),

  ("RIMS vs SPARC Reconciliation - Sub-ledger",
   "BBG vs RIMS/SPARC - Bonds (MTM)",
   "Bonds", "Principal/Position", "BS"),

  ("RIMS vs SPARC Reconciliation - Sub-ledger",
   "BBG/Sophis/GC Sparc vs Rims/Sparc - Bond Control",
   "Bonds", "Principal/Position", "BS"),

  ("BBG vs GCA - EM Bonds",
   "GC to GCA to BBG - POS + MtM",
   "Bonds", "MTM/Accrual", "PnL"),

  ("BBG vs RIMS/SPARC - Bonds (MTM)",
   "GC to GCA to BBG - POS + MtM",
   "Bonds", "Principal/Position", "BS"),

```

("BBG/Sophis/GC Sparc vs Rims/Sparc - Bond Control",
"BBG vs RIMS/SPARC - Bonds (MTM)",
"Bonds", "Cash", "BS"),

("GC to GCA to BBG - POS + MtM",
"HLR - Credit Flow",
"Bonds", "MTM/Accrual", "PnL"),

("GC to GCA to BBG - POS + MtM",
"HLR - Credit Flow",
"Bond Coupon", "Cash", "PnL"),

("HLR - Credit Flow",
"CRI COMPTA Rec - Bonds",
"Bonds", "Cash", "PnL"),

— MTN chain —————

("Summit/XFOS vs SPARC - MTNs MTM rec (Rates/Credits)",
"RIMS/Sparc vs HUB Cash Contra Rec",
"MTN", "Principal/Position", "BS"),

("Summit/XFOS vs SPARC - MTNs MTM rec (Rates/Credits)",
"HLR - Credit Flow",
"MTN", "MTM/Accrual", "PnL"),

("Summit/XFOS vs SPARC - MTNs Cash rec (Rates/Credits)",
"SPARC Netted reconciliation",
"MTN", "Cash", "BS"),

("RIMS/Sparc vs HUB Cash Contra Rec",
"SPARC Netted reconciliation",
"MTN", "Cash", "BS"),

("HLR - Credit Flow",
"CRI COMPTA Rec - Bonds",
"MTN", "Cash", "PnL"),

— RATES / IRD chain —————

("Summit vs Sparc - IRD MTM",
"HUB vs Summit/Sparc - IRD Contra",
"IRD", "MTM/Accrual", "PnL"),

("HUB vs Summit/Sparc - IRD Contra",
"HUB vs Mainframe - Fees & Interest",
"IRD", "Cash", "BS"),

("HUB vs Mainframe - Fees & Interest",

"HLR Rates",
"IRD", "Cash", "PnL"),

("HLR Rates",
"CRI COMPTA Rec - Derivatives",
"IRD", "MTM/Accrual", "PnL"),

— REPO chain —————

("Martini vs SPARC - Repo (MTM, Principal, BvB Fees)",
"Martini/Sparc vs Rims/Sparc - Repo Control",
"Repo", "Principal/Position", "BS"),

("Martini/Sparc vs Rims/Sparc - Repo Control",
"SPARC Netted reconciliation",
"Repo", "Cash", "BS"),

("SPARC Netted reconciliation",
"HUB vs Mainframe - Fees & Interest",
"Repo", "Cash", "BS"),

("HUB vs Mainframe - Fees & Interest",
"P&L Rec - REPO",
"Repo", "Cash", "PnL"),

("P&L Rec - REPO",
"CRI COMPTA Rec - REPO",
"Repo", "Cash", "PnL"),

— CREDITS / CDS chain —————

("GC vs DTP - CDS - Credit Centralised",
"GCA vs DTP - CRD Control",
"CDS", "Principal/Position", "BS"),

("GCA vs DTP - CRD Control",
"DTP vs HUB - CRD Contra",
"CDS", "Cash", "BS"),

("DTP vs HUB - CRD Contra",
"HLR - SIP",
"CDS", "Cash", "PnL"),

("XFOS vs DTP - TRS (MTM, Cash)",
"GCA vs DTP - CRD Control",
"TRS", "MTM/Accrual", "PnL"),

("Sentry vs LoanIQ - Secondary Loans",
"HLR - SIP",


```

"Loans", "Principal/Position", "BS"),

("HLR - SIP",
"CRI COMPTA Rec - CDS",
"Credits", "Cash", "PnL"),

# — FX OPTIONS chain —————
("Murex vs Sparc - FXO IRD (Cash, MTM)",
"Murex vs FTP (Balance Sheet, PL) - FX Options",
"FX Options", "MTM/Accrual", "PnL"),

("Murex vs FTP (Balance Sheet, PL) - FX Options",
"P&L Rec - FXO",
"FX Options", "MTM/Accrual", "PnL"),

("P&L Rec - FXO",
"CRI COMPTA Rec - FXO",
"FX Options", "Cash", "PnL"),

# — CONVERTIBLE BONDS chain —————
("SOPHIS vs FTP - Convertible Bonds",
"HLR - Credit Flow",
"Convertible Bonds", "MTM/Accrual", "PnL"),

("HLR - Credit Flow",
"CRI COMPTA Rec - Bonds",
"Convertible Bonds", "Cash", "PnL"),

# — FUTURES / ETD chain —————
("BBG vs XTP - Futures",
"Summit vs XTP - Futures (MTM)",
"Futures", "Principal/Position", "BS"),

("Summit vs XTP - Futures (MTM)",
"HLR Rates",
"Futures", "MTM/Accrual", "PnL"),

("HLR Rates",
"CRI COMPTA Rec - ETD CC",
"Futures", "Cash", "PnL"),

# — MONEY MARKET chain —————
("Summit vs Treats - MM (MTM, Notional)",
"SPARC Netted reconciliation",
"MM", "Principal/Position", "BS"),

("SPARC Netted reconciliation",

```

```

    "HUB vs Mainframe - Fees & Interest",
    "MM", "Cash", "BS"),

("HUB vs Mainframe - Fees & Interest",
 "CRI COMPTA Rec - MMK",
 "MM", "Cash", "BS"),

# — EM BONDS / EM SWAPS chain —————
("XFOS vs Treats - EM SWAPS (MTM, Cash Rec)",
 "GC to GCA to BBG - POS + MtM",
 "EM Swaps", "MTM/Accrual", "PnL"),

("GC to GCA to BBG - POS + MtM",
 "HLR - Credit Flow",
 "EM Swaps", "Cash", "PnL"),
]

# =====
# SECTION 4 — LOAD & STANDARDISE MI DATA
# =====

def load_data() -> pd.DataFrame:
    """Load the MI report. Falls back to synthetic demo data if no file is set."""
    if DF_PATH:
        ext = DF_PATH.rsplit(".", 1)[-1].lower()
        if ext == "csv":
            df = pd.read_csv(DF_PATH)
        elif ext in ("parquet", "pq"):
            df = pd.read_parquet(DF_PATH)
        elif ext in ("xlsx", "xls"):
            df = pd.read_excel(DF_PATH)
        else:
            raise ValueError(f"Unsupported file type: {ext}")
        print(f"[INFO] Loaded {len(df):,} rows from {DF_PATH}")
        return df
    else:
        print("[INFO] No DF_PATH set — generating synthetic demo data.")
        return _synthetic_demo()

def _synthetic_demo() -> pd.DataFrame:
    """Generate realistic synthetic MI data for all GDM recs."""
    rng = np.random.default_rng(42)
    rows = []
    recs = [r["rec"] for r in GDM_RECS]
    products = ["Bonds", "MTN", "IRD", "Repo", "CDS", "FX Options",
                "Futures", "MM", "EM Bonds", "Convertible Bonds", "TRS"]
    teams = ["Rates", "Credits", "EMNY", "Bonds", "Rates EM LATAM"]

```

```

break_types = ["Principal/Position", "MTM/Accrual", "Cash"]
account_groups = ["BS", "PnL"]
entities = ["ENTITY_A", "ENTITY_B", "ENTITY_C", "ENTITY_D", "ENTITY_E"]

for month_offset in [0, 1, 2]: # 3 months
    base_date = pd.Timestamp("2024-01-01") + pd.DateOffset(months=month_offset)
    for rec_info in GDM_RECS:
        # Simulate break count per rec per month (weighted by precedence)
        n_breaks = rng.integers(5, 60)
        for _ in range(n_breaks):
            trade_ref = f"TR{rng.integers(1000, 9999)}"
            entity = rng.choice(entities)
            product = rng.choice(rec_info["asset_classes"])
            bt = rng.choice(rec_info["break_types"])
            ag = rng.choice(rec_info["account_groups"])
            team = rng.choice(rec_info["teams"])
            amount = rng.normal(500_000, 2_000_000)
            jnl_count = rng.integers(0, 4) # journals posted for this break
            rows.append({
                COL["date"]: base_date + pd.Timedelta(days=int(rng.integers(1, 31))),
                COL["entity"]: entity,
                COL["asset_class"]: product,
                COL["product"]: product,
                COL["trade_ref"]: trade_ref,
                COL["rec_name"]: rec_info["rec"],
                COL["break_amount"]: round(amount, 2),
                COL["break_type"]: bt,
                COL["account_group"]: ag,
                COL["team"]: team,
                COL["ccy"]: rng.choice(["GBP", "USD", "EUR"]),
                "Journal Count": jnl_count,
                "Is Root Cause": rng.choice([True, False], p=[0.35, 0.65])
            })

return pd.DataFrame(rows)

```

```

def standardise(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df[COL["date"]] = pd.to_datetime(df[COL["date"]], errors="coerce")
    for key in ["entity", "asset_class", "product", "trade_ref", "rec_name",
                "break_type", "account_group", "team"]:
        col = COL[key]
        if col in df.columns:
            df[col] = df[col].astype(str).str.strip().fillna("UNKNOWN")
    df[COL["break_amount"]] = pd.to_numeric(
        df[COL["break_amount"]], errors="coerce"
    )

```

```

    ).fillna(0)
    df["rec_rank"] = df[COL["rec_name"]].map(REC_RANK).fillna(999).astype(int)
    return df

# =====
# SECTION 5 – JOURNAL POSTING ANALYSIS
# =====

def journal_posting_analysis(df: pd.DataFrame) -> dict[str, pd.DataFrame]:
    """
    For every rec break in the MI report, compute:
    - Total breaks
    - Total journals posted (irrespective of root cause)
    - Estimated unnecessary journals (inherited breaks)
    - Potential saving if root cause identified
    """
    rec_col = COL["rec_name"]
    amt_col = COL["break_amount"]
    bt_col = COL["break_type"]
    ag_col = COL["account_group"]
    team_col = COL["team"]

    # — Per-rec journal summary —————
    jnl_col = "Journal Count" if "Journal Count" in df.columns else None

    agg_dict: dict = {
        "Total Breaks": (rec_col, "count"),
        "Total Amount GBP": (amt_col, "sum"),
        "Unique Entities": (COL["entity"], "nunique"),
        "Unique Products": (COL["product"], "nunique"),
    }

    if jnl_col:
        agg_dict["Total Journals Posted"] = (jnl_col, "sum")
        agg_dict["Avg Journals Per Break"] = (jnl_col, "mean")

    rec_summary = (
        df.groupby(rec_col)
        .agg(**agg_dict)
        .reset_index()
        .rename(columns={rec_col: "Rec Name"})
    )

    # Enrich with metadata
    rec_summary["Precedence"] = rec_summary["Rec Name"].map(
        lambda r: REC_META.get(r, {}).get("precedence", 99)
    )

    rec_summary["TLM Divide"] = rec_summary["Rec Name"].map(

```

```

        lambda r: REC_META.get(r, {}).get("divide", "UNKNOWN")
    )
    rec_summary["TLM Stage"] = rec_summary["Rec Name"].map(
        lambda r: REC_META.get(r, {}).get("tlm_stage", "UNKNOWN")
    )
    rec_summary["Frequency"] = rec_summary["Rec Name"].map(
        lambda r: REC_META.get(r, {}).get("frequency", "UNKNOWN")
    )

    # Downstream recs – how many edges leave this rec in the propagation graph?
    downstream_count = defaultdict(int)
    for frm, to, *_ in PROPAGATION_EDGES:
        downstream_count[frm] += 1
    rec_summary["Downstream Rec Count"] = rec_summary["Rec Name"].map(
        lambda r: downstream_count.get(r, 0)
    )

    # Is this a known root cause rec? (has outgoing edges)
    rec_summary["Is Known Root Cause Rec"] = (
        rec_summary["Downstream Rec Count"] > 0
    )

    # Estimate inherited (false positive) breaks:
    # Recs at precedence > 2 that have at least one upstream feeder are
    # likely to have some inherited breaks (~40% based on your 3-month analysis)
    # Adjust INHERITED_RATE to your validated figure.
    INHERITED_RATE = 0.40

    # Upstream feeder check
    has_upstream = set(to for _, to, *_ in PROPAGATION_EDGES)
    rec_summary["Has Upstream Feeder"] = rec_summary["Rec Name"].isin(has_upstream)

    rec_summary["Est Inherited Breaks"] = (
        rec_summary.apply(
            lambda row: int(row["Total Breaks"] * INHERITED_RATE)
            if row["Has Upstream Feeder"] else 0,
            axis=1,
        )
    )

    if jnl_col:
        avg_jnl = (
            rec_summary["Total Journals Posted"] /
            rec_summary["Total Breaks"].clip(lower=1)
        )
        rec_summary["Est Suppressible Journals"] = (
            (rec_summary["Est Inherited Breaks"] * avg_jnl).round(0).astype(int)

```

```

    )
    rec_summary["Journal Saving %"] = (
        (rec_summary["Est Suppressible Journals"] /
         rec_summary["Total Journals Posted"].clip(lower=1) * 100)
        .round(1)
    )

rec_summary = rec_summary.sort_values(["Precedence", "Rec Name"])

# — Break-type × account-group split —————
if bt_col in df.columns and ag_col in df.columns:
    bt_ag = (
        df.groupby([rec_col, bt_col, ag_col])
        .agg(
            Break_Count=(rec_col, "count"),
            Total_Amount=(amt_col, "sum"),
        )
        .reset_index()
        .rename(columns={rec_col: "Rec Name",
                          bt_col: "Break Type",
                          ag_col: "Account Group"})
    )
else:
    bt_ag = pd.DataFrame()

# — Team ownership summary —————
if team_col in df.columns:
    team_summary = (
        df.groupby([rec_col, team_col])
        .agg(
            Break_Count=(rec_col, "count"),
            Total_Amount=(amt_col, "sum"),
        )
        .reset_index()
        .rename(columns={rec_col: "Rec Name", team_col: "Team"})
    )
else:
    team_summary = pd.DataFrame()

return {
    "rec_summary": rec_summary,
    "break_type_ag": bt_ag,
    "team_summary": team_summary,
}

```

```

# SECTION 6 - LINKAGE DIAGRAM BUILDER
# Generates an interactive HTML network diagram matching your image style:
#   GREEN   node = Reconciliation
#   BROWN   node = Product
#   BLUE    node = Break Type + Account Group
#   ORANGE  node = Team
#   WHITE   node = Asset class root / GDM
# =====
# Colour constants (hex) - matching your diagram exactly
NODE_COLORS = {
    "root":      "#FFFFFF",    # white - GDM / asset class root
    "rec":       "#2D6A2D",    # green - reconciliation
    "product":   "#7B2A2A",    # brown - product label
    "break_type": "#1A6080",    # blue - break type / account group
    "team":      "#E07B20",    # orange - team
}
EDGE_COLORS = {
    "BS":        "#5B9BD5",    # balance sheet flow - blue
    "PnL":       "#70AD47",    # P&L flow - green
    "unknown":   "#AAAAAA",
}

def build_linkage_diagram(
    filter_asset_class: str | None = None,
    filter_product: str | None = None,
    output_html: str = "gdm_linkage_diagram.html",
) -> None:
    """
    Build a pyvis interactive network matching the style in your image.

    Parameters
    -----
    filter_asset_class : if given, only show recs touching this asset class
    filter_product      : if given, only show edges for this product
    output_html         : output file path
    """
    if not PYVIS_OK:
        print("[SKIP] pyvis not available - cannot generate HTML diagram.")
        return

    net = Network(
        height="900px", width="100%",
        bgcolor="#1A1A2E",
        font_color="#FFFFFF",
        directed=True,
        notebook=False,

```

```

)
net.set_options("""
{
    "physics": {
        "enabled": true,
        "hierarchicalRepulsion": {
            "centralGravity": 0.0,
            "springLength": 180,
            "springConstant": 0.01,
            "nodeDistance": 160
        },
        "solver": "hierarchicalRepulsion"
    },
    "layout": {
        "hierarchical": {
            "enabled": true,
            "direction": "UD",
            "sortMethod": "directed",
            "nodeSpacing": 200,
            "levelSeparation": 160
        }
    },
    "edges": {
        "arrows": { "to": { "enabled": true, "scaleFactor": 0.8 } },
        "smooth": { "type": "cubicBezier", "forceDirection": "vertical" },
        "width": 2
    },
    "nodes": {
        "shape": "box",
        "borderWidth": 2,
        "font": { "size": 11, "face": "Courier New" }
    },
    "interaction": {
        "hover": true,
        "tooltipDelay": 200
    }
}
""")

```

```

added_nodes: set[str] = set()

```

```

def safe_node(node_id: str, label: str, color: str,
              level: int = 0, title: str = "") -> None:
    if node_id not in added_nodes:
        net.add_node(
            node_id, label=label, color=color,
            level=level,

```



```

        title=title or label,
        font={"color": "#FFFFFF", "size": 11},
        margin=8,
    )
    added_nodes.add(node_id)

# — Root node — GDM —————
safe_node("GDM", "GDM", NODE_COLORS["root"], level=0,
        title="GDM Asset Class Root")

# — Asset class grouping nodes —————
ASSET_CLASS_GROUPS = [
    "Bonds", "MTN", "IRD", "Repo", "Credits / CDS",
    "FX Options", "Futures / ETD", "Money Market",
    "EM Bonds / Swaps", "Convertible Bonds",
]
for ac in ASSET_CLASS_GROUPS:
    if filter_asset_class and filter_asset_class.lower() not in ac.lower():
        continue
    safe_node(f"AC_{ac}", ac, NODE_COLORS["root"], level=1,
            title=f"Asset class: {ac}")
    net.add_edge("GDM", f"AC_{ac}",
            color="#CCCCCC", width=1, dashes=True)

# — Map each rec to its primary asset class group —————
REC_TO_AC = {
    "RIMS vs SPARC Reconciliation - Sub-ledger": "Bonds",
    "BBG vs GCA - EM Bonds": "Bonds",
    "BBG vs RIMS/SPARC - Bonds (MTM)": "Bonds",
    "BBG/Sophis/GC Sparc vs Rims/Sparc - Bond Control": "Bonds",
    "GC to GCA to BBG - POS + MtM": "Bonds",
    "HLR - Credit Flow": "Bonds",
    "CRI COMPTA Rec - Bonds": "Bonds",
    "Summit/XFOS vs SPARC - MTNs MTM rec (Rates/Credits)": "MTN",
    "Summit/XFOS vs SPARC - MTNs Cash rec (Rates/Credits)": "MTN",
    "Summit vs Sparc - IRD MTM": "IRD",
    "HUB vs Summit/Sparc - IRD Contra": "IRD",
    "HLR Rates": "IRD",
    "CRI COMPTA Rec - Derivatives": "IRD",
    "Martini vs SPARC - Repo (MTM, Principal, BvB Fees)": "Repo",
    "Martini/Sparc vs Rims/Sparc - Repo Control": "Repo",
    "P&L Rec - REPO": "Repo",
    "CRI COMPTA Rec - REPO": "Repo",
    "GC vs DTP - CDS - Credit Centralised": "Credits / CDS",
    "GCA vs DTP - CRD Control": "Credits / CDS",
    "DTP vs HUB - CRD Contra": "Credits / CDS",
    "Sentry vs LoanIQ - Secondary Loans": "Credits / CDS"
}

```

```

"XFOS vs DTP - TRS (MTM, Cash)": "Credits / CDS",
"HLR - SIP": "Credits / CDS",
"CRI COMPTA Rec - CDS": "Credits / CDS",
"Murex vs Sparc - FXO IRD (Cash, MTM)": "FX Options",
"Murex vs FTP (Balance Sheet, PL) - FX Options": "FX Options",
"P&L Rec - FXO": "FX Options",
"CRI COMPTA Rec - FXO": "FX Options",
"BBG vs XTP - Futures": "Futures / ETD",
"Summit vs XTP - Futures (MTM)": "Futures / ETD",
"CRI COMPTA Rec - ETD CC": "Futures / ETD",
"Summit vs Treats - MM (MTM, Notional)": "Money Market",
"CRI COMPTA Rec - MMK": "Money Market",
"XFOS vs Treats - EM SWAPS (MTM, Cash Rec)": "EM Bonds / Swap",
"SOPHIS vs FTP - Convertible Bonds": "Convertible Bo",
"BvB Fees Rec": "Bonds",
"GC vs Bloomberg - Credit Linked Notes": "Credits / CDS",
"NCOMs vs Treats - Collateral (Position, Interest Accrual)": "Repo",
"Summit vs Treats - FX (MTM, Cash)": "FX Options",
"SPARC Netted reconciliation": "Bonds",
"RIMS/Sparc vs HUB Cash Contra Rec": "Bonds",
"HUB vs Mainframe - Fees & Interest": "IRD",
"CRI COMPTA Rec - OTC CC": "IRD",
"CRI COMPTA Rec - Margin Call": "IRD",
"CRI COMPTA Rec - MMK": "Money Market"
}

```

```

# — Filter edges by product if requested —————
edges_to_draw = [

```

```

    e for e in PROPAGATION_EDGES
    if filter_product is None or e[2] == filter_product
]

```

```

# — Add rec nodes + product/break/team satellite nodes —————
for frm, to, product, break_type, account_group in edges_to_draw:

```

```

    for rec_name in [frm, to]:
        meta = REC_META.get(rec_name, {})
        prec = meta.get("precedence", 99)
        level = prec + 1 # level 0 = GDM root, 1 = asset class
        ac = REC_TO_AC.get(rec_name, "")

```

```

        if filter_asset_class and ac != filter_asset_class:
            continue

```

```

        # Rec node (green)

```

```

        tooltip = (
            f"Rec: {rec_name}\n"
            f"Precedence: {prec} ({meta.get('divide', '')})\n"

```

```

        f"Frequency: {meta.get('frequency','')}\n"
        f"TLM Stage: {meta.get('tlm_stage','')}"
    )
    safe_node(rec_name, _wrap(rec_name, 25),
              NODE_COLORS["rec"], level=level, title=tooltip)

    # Connect rec to asset class group node
    if ac and f"AC_{ac}" in added_nodes:
        if ("GDM", rec_name) not in added_nodes:
            net.add_edge(f"AC_{ac}", rec_name,
                        color="#AAAAAA", width=1, dashes=True)
            added_nodes.add(("GDM", rec_name))

    # Product node (brown) - satellite
    prod_id = f"PROD_{rec_name}_{product}"
    safe_node(prod_id, product, NODE_COLORS["product"],
              level=level, title=f"Product: {product}")
    if (prod_id, rec_name) not in added_nodes:
        net.add_edge(prod_id, rec_name, color=NODE_COLORS["product"],
                    width=1, dashes=True, arrows="")
        added_nodes.add((prod_id, rec_name))

    # Break type + account group (blue) - satellite
    bt_id = f"BT_{rec_name}_{break_type}_{account_group}"
    bt_label = f"Break: {break_type}\nAG:{account_group}"
    safe_node(bt_id, bt_label, NODE_COLORS["break_type"],
              level=level, title=f"Break Type: {break_type} | AG: {account_group}")
    if (bt_id, rec_name) not in added_nodes:
        net.add_edge(rec_name, bt_id, color=NODE_COLORS["break_type"],
                    width=1, dashes=True, arrows="")
        added_nodes.add((bt_id, rec_name))

    # Team nodes (orange)
    teams = meta.get("teams", [])
    for team in teams:
        team_id = f"TEAM_{team}"
        safe_node(team_id, team, NODE_COLORS["team"],
                  level=level + 1, title=f"Owning team: {team}")
        if (bt_id, team_id) not in added_nodes:
            net.add_edge(bt_id, team_id,
                        color=NODE_COLORS["team"], width=1)
            added_nodes.add((bt_id, team_id))

    # Main propagation edge (rec → rec)
    edge_col = EDGE_COLORS.get(account_group, EDGE_COLORS["unknown"])
    edge_title = (
        f"Product: {product}\n"

```

```

        f"Break Type: {break_type}\n"
        f"Account Group: {account_group}"
    )
    net.add_edge(
        frm, to,
        color=edge_col,
        width=3,
        title=edge_title,
        label=f"{product} | {account_group}",
    )

    out_path = os.path.join(OUTPUT_DIR, output_html)
    net.write_html(out_path)
    print(f"[OK] Linkage diagram saved → {out_path}")

def _wrap(text: str, width: int) -> str:
    """Wrap long rec names at word boundaries."""
    words = text.split()
    lines, line = [], ""
    for w in words:
        if len(line) + len(w) + 1 > width and line:
            lines.append(line)
            line = w
        else:
            line = f"{line} {w}".strip()
    if line:
        lines.append(line)
    return "\n".join(lines)

# =====
# SECTION 7 – PLOTLY JOURNAL POSTING CHARTS
# =====

def build_journal_charts(rec_summary: pd.DataFrame) -> None:
    """Generate Plotly HTML charts for journal posting analysis."""
    if not PLOTLY_OK:
        print("[SKIP] plotly not available – skipping charts.")
        return

    # — Chart 1: Total breaks per rec, coloured by precedence —————
    fig1 = px.bar(
        rec_summary.sort_values("Total Breaks", ascending=True).tail(30),
        x="Total Breaks",
        y="Rec Name",
        color="TLM Divide",
        orientation="h",
    )

```

```

        title="Total Breaks per Rec (Current MI Report)",
        labels={"Total Breaks": "Break Count", "Rec Name": ""},
        color_discrete_sequence=px.colors.qualitative.Dark24,
        height=800,
    )
    fig1.update_layout(
        paper_bgcolor="#1A1A2E", plot_bgcolor="#1A1A2E",
        font=dict(color="#E0E0E0", size=11),
        legend=dict(bgcolor="#0F0F1E", bordercolor="#333"),
        title_font=dict(size=14),
    )
    p1 = os.path.join(OUTPUT_DIR, "chart_breaks_per_rec.html")
    fig1.write_html(p1)
    print(f"[OK] Chart 1 → {p1}")

# — Chart 2: Journals posted per rec (if data available) —————
if "Total Journals Posted" in rec_summary.columns:
    fig2 = px.bar(
        rec_summary.sort_values("Total Journals Posted", ascending=True).tail
        x="Total Journals Posted",
        y="Rec Name",
        color="TLM Divide",
        orientation="h",
        title="Total Journals Posted per Rec Break (Current MI – Irrespective
        labels={"Total Journals Posted": "Journal Count", "Rec Name": ""},
        color_discrete_sequence=px.colors.qualitative.Pastel,
        height=800,
    )
    fig2.update_layout(
        paper_bgcolor="#1A1A2E", plot_bgcolor="#1A1A2E",
        font=dict(color="#E0E0E0", size=11),
    )
    p2 = os.path.join(OUTPUT_DIR, "chart_journals_per_rec.html")
    fig2.write_html(p2)
    print(f"[OK] Chart 2 → {p2}")

# — Chart 3: Actual vs suppressible journals —————
if "Est Suppressible Journals" in rec_summary.columns:
    top = rec_summary.nlargest(20, "Total Journals Posted")
    fig3 = go.Figure(data=[
        go.Bar(name="Journals Posted",
            x=top["Rec Name"], y=top["Total Journals Posted"],
            marker_color="#4472C4"),
        go.Bar(name="Est. Suppressible (Root Cause Identified)",
            x=top["Rec Name"], y=top["Est Suppressible Journals"],
            marker_color="#ED7D31"),
    ])

```

```

fig3.update_layout(
    barmode="overlay",
    title="Journal Posting Reduction Opportunity – Top 20 Recs",
    paper_bgcolor="#1A1A2E", plot_bgcolor="#1A1A2E",
    font=dict(color="#E0E0E0", size=10),
    xaxis_tickangle=-45,
    height=600,
    legend=dict(bgcolor="#0F0F1E"),
)
p3 = os.path.join(OUTPUT_DIR, "chart_journal_saving.html")
fig3.write_html(p3)
print(f"[OK] Chart 3 → {p3}")

# — Chart 4: Break type × account group heatmap —————
if PLOTLY_OK:
    prec_counts = (
        rec_summary.groupby(["Precedence", "TLM Divide"])
        ["Total Breaks"].sum().reset_index()
    )
    fig4 = px.sunburst(
        rec_summary,
        path=["TLM Divide", "Frequency", "Rec Name"],
        values="Total Breaks",
        title="GDM Break Distribution – Precedence → Frequency → Rec",
        color="Precedence",
        color_continuous_scale="Blues",
        height=700,
    )
    fig4.update_layout(
        paper_bgcolor="#1A1A2E",
        font=dict(color="#E0E0E0"),
    )
    p4 = os.path.join(OUTPUT_DIR, "chart_break_sunburst.html")
    fig4.write_html(p4)
    print(f"[OK] Chart 4 → {p4}")

# =====
# SECTION 8 – EXCEL EXPORT
# =====

def export_excel(results: dict[str, pd.DataFrame]) -> None:
    """Write all analysis tables to a colour-formatted Excel workbook."""
    out_path = os.path.join(OUTPUT_DIR, "gdm_journal_analysis.xlsx")
    try:
        with pd.ExcelWriter(out_path, engine="openpyxl") as writer:
            for sheet_name, df_out in results.items():
                if df_out is None or len(df_out) == 0:

```

```

        continue
    safe_name = sheet_name[:31]
    df_out.to_excel(writer, sheet_name=safe_name, index=False)

    ws = writer.sheets[safe_name]
    # Auto-fit columns
    for col_cells in ws.columns:
        max_len = max(
            (len(str(c.value)) for c in col_cells if c.value),
            default=8,
        )
        ws.column_dimensions[col_cells[0].column_letter].width = (
            min(max_len + 2, 60)
        )

    print(f"[OK] Excel workbook → {out_path}")
except ImportError:
    print("[WARN] openpyxl not installed – Excel export skipped. pip install

# =====
# SECTION 9 – MAIN ENTRY POINT
# =====

def main() -> None:
    print("\n" + "="*65)
    print("GDM BREAK LINKAGE DIAGRAM + JOURNAL POSTING ANALYSIS")
    print("="*65)

    # — 1. Load data —————
    df_raw = load_data()
    df = standardise(df_raw)
    print(f"\nData: {len(df):,} rows | {df[COL['rec_name']].nunique()} unique rec

    # — 2. Journal posting analysis —————
    print("\n[STEP 1] Running journal posting analysis...")
    results = journal_posting_analysis(df)
    rec_sum = results["rec_summary"]

    print("\n— Top 10 recs by break count —")
    cols_show = ["Rec Name", "Precedence", "TLM Divide", "Frequency",
                  "Total Breaks", "Total Amount GBP"]
    if "Total Journals Posted" in rec_sum.columns:
        cols_show += ["Total Journals Posted", "Est Suppressible Journals",
                      "Journal Saving %"]
    print(rec_sum.nlargest(10, "Total Breaks")[cols_show].to_string(index=False))

    if "Total Journals Posted" in rec_sum.columns:
        total_jnl = rec_sum["Total Journals Posted"].sum()

```

```

        total_sup = rec_sum.get("Est Suppressible Journals", pd.Series([0])).sum
        print(f"\nTotal journals posted (all recs): {int(total_jnl):,}")
        print(f"Estimated suppressible journals: {int(total_sup):,} "
              f"({int(total_sup)/max(total_jnl,1)*100:.1f}% saving)")

# — 3. Build diagrams —————
print("\n[STEP 2] Building linkage diagrams...")

# Full GDM diagram
build_linkage_diagram(
    output_html="gdm_linkage_full.html",
)

# Per-asset-class diagrams
for ac in ["Bonds", "MTN", "IRD", "Repo", "Credits / CDS",
          "FX Options", "Futures / ETD"]:
    safe = ac.replace("/", "_").replace(" ", "_")
    build_linkage_diagram(
        filter_asset_class=ac,
        output_html=f"gdm_linkage_{safe}.html",
    )

# — 4. Charts —————
print("\n[STEP 3] Building journal posting charts...")
build_journal_charts(rec_sum)

# — 5. Excel export —————
print("\n[STEP 4] Exporting to Excel...")
export_excel({
    "Rec Journal Summary": rec_sum,
    "Break Type x AG": results["break_type_ag"],
    "Team Ownership": results["team_summary"],
    "Propagation Edges": pd.DataFrame(
        PROPAGATION_EDGES,
        columns=["From Rec", "To Rec", "Product", "Break Type", "Account Group"],
    ),
})

print("\n[DONE] All outputs written to:", os.path.abspath(OUTPUT_DIR))
print("\nFiles generated:")
for f in sorted(os.listdir(OUTPUT_DIR)):
    size = os.path.getsize(os.path.join(OUTPUT_DIR, f))
    print(f" {f:<55} {size/1024:.0f} KB")

if __name__ == "__main__":
    main()

```


